



Algebra Relazionale

Prof Ionel Eduard Stan
A.A. 2025/2026

Contesto della lezione



1. Ripasso ultrarapido della lezione scorsa
2. Operatore di divisione
3. Aggregazione γ e bag-semantics essenziale
4. Outer-join (join esterne) e valori NULL
5. Da SQL ad algebra relazionale (traduzione, sottoquery, viste)
6. Ottimizzazione cost-based (join order)
7. Esercizi avanzati e riepilogo

Ripasso della lezione 1

- 
- Operatori unari: selezione (σ), proiezione (π), ridenominazione (ρ)
 - Operatori binari: prodotto cartesiano ($\times$), theta-join ($\bowtie_{\theta}$), natural-join ($\bowtie$)
 - Operatori insiemistici: unione ($\cup$), intersezione ($\cap$), differenza ($-$)
 - Abbiamo visto esempi per ognuno di questi operatori
 - Alberi di esecuzione ed alcune equivalenze tra espressioni algebriche (es., push-down di σ)
 - Infine, abbiamo concluso con alcuni esercizi

Divisione ($\div$): Intuizione e definizione

- 
- **Intuizione:** domande di tipo universale
 - Esempio: Impiegati che hanno svolto **tutti** i progetti chiave
 - Schema ricorrente: $R(X,Y)$ e $S(Y) \Rightarrow$ risultato sui soli attributi X
 - **Definizione formale:** $R \div S = \{ t[X] \mid t \in R \text{ e } \forall s \in S (t[X], s[Y]) \in R \}$
 - In uscita solo gli X che “coprono” ogni tupla di S
 - La divisione si può **derivare** da altri operatori. In effetti, siano $R(X,Y)$ e $S(Y)$ due relazioni, allora:

$$R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

- Dunque, è un operatore non primitivo, ma utile in scrittura

Divisione ($\div$): Esempio

- Schema:

IMPIEGATO(CodImp, CodProg)

PROG_OBBLIGATORIO(CodProg)

- **Espressione:** $\pi_{\text{CodImp}}(\text{IMPIEGATO} \div \text{PROG_OBBLIGATORIO})$
- **Risultato:** gli impiegati che compaiono con **ogni** CodProg in PROG_OBBLIGATORIO

Aggregazione (γ): Sintassi

- 
- Come in SQL, anche in algebra relazionale abbiamo la possibilità di aggregare i risultati di interrogazioni usando l'**operatore di aggregazione γ**
 - **Sintassi:** $\gamma_{G; f_1(A_1) \rightarrow F_1, \dots} (R)$
 - G sono gli attributi di raggruppamento
 - Le funzioni f sono funzione di aggregazione (COUNT, SUM, MAX, MIN, AVG)
 - $f_1(A_1) \rightarrow F_1$ vuol dire la funzione f_1 applicata all'attributo A_1 e rinominiamo il risultato come F_1
 - Se non volessimo usare la rinomina dell'operatore γ , possiamo sempre usare l'operatore di ridenominazione ρ (visto la scorsa lezione)

Aggregazione (γ): Esempio

- Schema:

IMPIEGATO(CodImp, Dip, Stip)

- **Richiesta:** trovare gli stipendi medi per ogni dipartimento
- **Espressione:** $\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$
- **Risultato:** una tupla per Dip con colonna Stip_medio

Dip	Stip_medio
1	15000
2	5000

$\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$

CodImp	Dip	Stip
1	1	10000
2	1	20000
3	2	5000
4	2	3000
5	2	7000

IMPIEGATO

Bag-semantics essenziale



- Ricordiamo che SQL opera su **bag**, cioè ci sono duplicati
- L'algebra relazionale invece opera su insiemi, cioè non ci sono duplicati
- y su COUNT differisce se ci sono duplicati
- Questa è una distinzione fondamentale tra SQL e algebra relazionale, sebbene nella pratica ci sono modi per by-passare questo “problema” (es., dipende dalla modellazione dei dati)

Valori NULL nel modello relazionale

- NULL rappresenta “valore sconosciuto / non applicabile”
- Quando NULL compare in un confronto, il risultato non è né vero né falso ma UNKNOWN
- Il modello segue la **logica a tre valori** (3VL–3-valued logic): TRUE (T), FALSE (F), UNKNOWN (U)

$\wedge$	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

$\vee$	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

$\neg$	
T	F
F	T
U	U

- Gli operatori di selezione (σ) considerano vero **solo** TRUE; FALSE e UNKNOWN vengono scartati
- I join interni ($\bowtie$) eliminano le tuple che producono UNKNOWN nell'espressione di uguaglianza

Join esterne: Concetto generale

- 
- Una **outer-join** (o **join esterna**) combina le tuple come un join interno e mantiene quelle che non trovano corrispondenze, riempiendo con NULL le colonne mancanti
 - Si definiscono tre varianti:
 - **Left outer join** $\bowtie_{\text{LEFT}}$: preserva tutte le tuple della relazione sinistra
 - **Right outer join** $\bowtie_{\text{RIGHT}}$: preserva tutte le tuple della relazione destra
 - **Full outer join** $\bowtie_{\text{FULL}}$: preserva entrambe le relazioni
 - Ogni outer-join utilizza la stessa logica di uguaglianza (theta-join) del join interno per determinare le corrispondenze

Left outer join ($\bowtie_{\text{LEFT}}$): Esempio



CodCli	Nome
1	Anna
2	Bruno
3	Carla

CLIENTE

CodOrd	CodCli	Importo
10	1	120
11	3	90

ORDINE

CodCli	Nome	CodOrd	Importo
1	Anna	10	120
2	Bruno	NULL	NULL
3	Carla	11	90

CLIENTE $\bowtie_{\text{LEFT CLIENTE.CodCli = ORDINE.CodCli}}$ ORDINE

Right outer join ($\bowtie_{\text{RIGHT}}$): Esempio



Matricola	Nome
501	Rossi

DOCENTE

CodCor	Docente	Titolo
BDD	501	Basi di Dati
DM	502	Data Mining

CORSO

Matricola	Nome	CodCor	Titolo
501	Rossi	BDD	Basi di Dati
NULL	NULL	DM	Data Mining

DOCENTE $\bowtie_{\text{RIGHT}}$ CORSO
RIGHT DOCENTE.Matricola = CORSO.Docente

Buone pratiche e insidie con NULL e outer-join

- 
- Predicato di filtro **dopo** l'outer-join possono eliminare tuple orfane se non si tiene conto dei NULL
 - L'outer-join **non** sono commutative in presenza di NULL: cambiare l'ordine di valutazione può produrre risultati diversi
 - L'equivalenza “ σ push-down” vale solo se il predicato non elimina tuple orfane necessarie alla semantica dell'outer-join
 - Attenzione a funzioni di aggregazione: COUNT(*) include le righe con NULL, mentre COUNT(X) ignora i NULL della colonna X specificata
 - I piani del DBMS materializzano spesso gli outer-join perché le tuple NULL-padded non possono essere generate a flusso finché non si sa se esiste un match

Dal linguaggio SQL all'algebra relazionale: Perché



- SQL è **dichiarativo**: l'utente indica cosa vuole, non come
- Il DBMS converte la query in una **espressione algebrica** e poi in un piano fisico
- Capire la corrispondenza **da SQL ad algebra relazionale** aiuta a leggere i piani e a ottimizzare interrogazioni complesse
- L'algebra relazionale rimane la “lingua franca” interna fra parser, ottimizzatore e motore di esecuzione

Mappa diretta fra clausole



SELECT (DISTINCT) $\Rightarrow \pi$ (o γ se ci sono aggregazioni)

FROM T1, T2 ... $\Rightarrow \times$ poi $\bowtie$ con le clausole ON

JOIN ... ON ... $\Rightarrow \bowtie$ (theta-join o natural-join)

WHERE predicati $\Rightarrow \sigma$

GROUP BY attributi $\Rightarrow \gamma$ (raggruppamento)

HAVING predicati $\Rightarrow \sigma$ sopra il risultato di γ

ORDER BY $\Rightarrow$ non espresso in algebra relazionale (ordinamento esterno)

- Alias/AS in SQL corrispondono a ρ (rinominazione)
- Sottoquery con IN / EXISTS si traducono combinando join, proiezioni, e differenze

Esempio: Join con filtro

- Schema:

IMPIEGATO(CodImp, Cognome, CodDip, Stipendio)

DIPARTIMENTO(CodDip, NomeDip)

- SQL:

```
SELECT i.Cognome, d.NomeDip
```

```
FROM IMPIEGATO i JOIN DIPARTIMENTO d on i.CodDip = d.CodDip
```

```
WHERE i.Stipendio > 3000;
```

- Espressione algebrica:

$$\pi_{\text{Cognome, NomeDip}}(\sigma_{\text{Stipendio} > 3000}(\text{IMPIEGATO} \bowtie_{\text{CodDip}} \text{DIPARTIMENTO}))$$

Esempio: Sotto-query con IN

- SQL:

```
SELECT CodImp, Cognome
```

```
FROM IMPIEGATO
```

```
WHERE CodImp IN (SELECT CodImp FROM PARTECIPAZIONE);
```

- **Espressione algebrica:**

$$\pi_{\text{CodImp, Cognome}}(\text{IMPIEGATO} \bowtie_{\text{CodImp}} \pi_{\text{CodImp}}(\text{PARTECIPAZIONE}))$$

- La proiezione sul solo CodImp garantisce che lo schema sia compatibile
- Il join filtra IMPIEGATO restituendo solo chi compare anche nella sottoquery

Esempio: Sotto-query con NOT IN

- SQL:

```
SELECT CodImp, Cognome
```

```
FROM IMPIEGATO
```

```
WHERE CodImp NOT IN (SELECT CodImp FROM PARTECIPAZIONE);
```

- Espressioni algebriche:

$$T1 \leftarrow \pi_{\text{CodImp, Cognome}}(\text{IMPIEGATO})$$
$$T2 \leftarrow \pi_{\text{CodImp, Cognome}}(\text{IMPIEGATO} \bowtie_{\text{CodImp}} \pi_{\text{CodImp}}(\text{PARTECIPAZIONE}))$$
$$R \leftarrow T1 - T2$$

- Prima si ottiene l'insieme completo di impiegati (T1)
- Poi si sottrae l'insieme di quelli che hanno corrispondenza nella sottoquery (T2)

Esempio: Sotto-query con EXISTS

- SQL:

```
SELECT d.NomeDip FROM DIPARTIMENTO d
```

```
WHERE EXISTS ( SELECT * FROM IMPIEGATO i
```

```
WHERE i.CodDip = d.CodDip AND i.Età < 30 );
```

- **Espressione algebrica:**

$$\pi_{\text{NomeDip}}(\text{DIPARTIMENTO} \bowtie_{\text{CodDip}} \sigma_{\text{Età} < 30}(\text{IMPIEGATO}))$$

- L'EXISTS diventa un join fra DIPARTIMENTO e l'insieme filtrato di IMPIEGATO
- Se non esiste nessun giovane in un dipartimento, nessuna tupla di quel dipartimento sopravvive al join

Esempio: Viste



- SQL:

```
CREATE VIEW GIOVANI AS
```

```
SELECT CodImp, Cognome, CodDip
```

```
FROM IMPIEGATO
```

```
WHERE Età < 30;
```

- **Espressione algebrica:**

$$\text{GIOVANI} \leftarrow \pi_{\text{CodImp, Cognome, CodDip}}(\sigma_{\text{Età} < 30}(\text{IMPIEGATO}))$$

- La vista è virtuale: la sostituzione (“expansion”) usa sempre l’espressione salvata
- Abbiamo già visto altri esempi in precedenza di viste (solo che non le avevamo ancora introdotte)
- Come simboli possiamo usare $\leftarrow$, $\rightarrow$, oppure $=$ (da intendersi con la semantica appena introdotta)

Unione, intersezione, differenza in SQL e algebra relazionale



SQL

Algebra relazionale (set semantics)

UNION

$\Rightarrow$

$R \cup S$

INTERSECT

$\Rightarrow$

$R \cap S$

EXCEPT / MINUS

$\Rightarrow$

$R - S$

- In algebra relazionale classica l'eliminazione dei duplicati è implicita
- In SQL serve la parola chiave DISTINCT o UNION senza ALL
- Gli operatori si possono combinare senza limitazioni, purché gli schemi siano compatibili (stessi attributi e domini)

In sintesi

- 
- $\sigma, \pi, \bowtie, \gamma, \cup, \cap, -, \rho$ bastano a tradurre gran parte di SQL, incluse sottoquery
 - Le sottoquery IN e EXISTS si esprimono con join e, se serve, set-difference per l'esclusione
 - Le viste sono semplicemente **espressioni algebriche denominate**; possono rimanere virtuali o essere materializzate con mantenimento incrementale
 - Padroneggiare la traduzione da SQL ad algebra relazionale rende leggibile qualsiasi piano logico che il DBMS produce

Ottimizzazione cost-based



- L'**ottimizzazione cost-based** è quella fase del DBMS che **sceglie il piano logico e fisico** più “economico” fra decine o centinaia di alternative valide
- Guida la scelta un **modello di costo** (I/O su disco, CPU, memoria)
- Lavora sull'albero di esecuzione derivato dalla algebra relazionale, applicando trasformazioni equivalenti

Statistiche e stima selettività



- Tabelle delle statistiche: cardinalità delle relazioni, numero di pagine, istogrammi di frequenza
- **Selettività di un predicato:** frazione di tuple attese dopo un filtro
- Stime accurate permettono di valutare in anticipo dimensioni dei risultati intermedi

Ordine dei join



- L'ordine in cui vengono eseguiti join, come abbiamo visto, è fondamentale per avere un piano (quasi) ottimo
- Strategie possibili:
 - **Left-deep:** join lineari producendo un flusso continuo di tuple, semplice da enumerare (pipelining efficiente)
 - **Bushy:** join su sotto-alberi paralleli, riduce la cardinalità intermedia (utile con molte tabelle)
- L'ottimizzatore esplora un sottoinsieme di piani possibili e seleziona quello con costo stimato minimo

Messaggi-chiave



- Il piano dipende **fortemente** dalle statistiche
- Statistiche obsolete danno piani sub-ottimali
- Ordinare i join in modo da filtrare prima le relazioni più selettive riduce i dati intermedi
- Comprendere la logica cost-based aiuta a scrivere query che il motore può ottimizzare con facilità

Esercizio: Divisione universale

- Schema:

ESAME(Matricola, CodCorso, Voto)

CORSO_OBBLIG(CodCorso) – corsi obbligatori

- **Traccia:** Elencare le matricole degli studenti che hanno superato (Voto ≥ 18) **tutti** i corsi obbligatori
- **Espressione algebrica:**

$T1 \leftarrow \sigma_{\text{Voto} \geq 18}(\text{ESAME})$

$\text{RISULTATO} \leftarrow \pi_{\text{Matricola}}(T1 \div \text{OBBLIG})$

- **Commenti:** La divisione restituisce solo le matricole che coprono l'intero insieme CORSO_OBBLIG filtrato sui voti validi

Esercizio: Aggregazione γ con join

- Schema:

PRODOTTO(PID, Categoria)

VENDITA(NumOrd, PID, Quantità, PrezzoUnit)

- **Traccia:** Trovare il **fatturato totale** (Quantità * PrezzoUnit) per categoria
- **Espressione algebrica:**

$\gamma_{\text{Categoria}; \text{SUM(Quantità*PrezzoUnit)} \rightarrow \text{Totale}} (\text{VENDITA} \bowtie \text{PRODOTTO})$

- **Commenti:**
 - Il join associa vendite al prodotto
 - L'operatore γ raggruppa per Categoria e somma il valore esteso

Esercizio: Left outer-join con media

- Schema:

CORSO(CodCorso, Titolo)

VALUTAZIONE(ID, CodCorso, Stelle) – le stelle vanno da 1 a 5

- **Traccia:** Mostrare tutti i corsi con la **media delle stelle**; se nessuna valutazione, la media rimane NULL
- **Espressione algebrica:**

$T1 \leftarrow \gamma_{\text{CodCorso}; \text{AVG}(\text{Stelle}) \rightarrow \text{Media}} (\text{VALUTAZIONE})$

$\text{RISULTATO} \leftarrow \text{CORSO} \bowtie_{\text{LEFT CORSO.CodCorso} = \text{T1.CodCorso}} T1$

- **Commenti:** Il left-outer join preserva i corsi senza valutazioni, riempiendo Media con NULL

Esercizio: Right outer-join su dati mancanti

- Schema:

RICERCATORE(RID, Nome)

PROGETTO(PID, Titolo, RID) – responsabile opzionale

- **Traccia:** Elencare Titolo progetto e Nome responsabile, mostrando NULL se il responsabile non è registrato
- **Espressione algebrica:**

$T1 \leftarrow \text{RICERCATORE} \bowtie_{\text{RIGHT RICERCATORE.RID} = \text{PROGETTO.RID}} \text{PROGETTO}$

$\text{RISULTATO} \leftarrow \pi_{\text{Titolo, Nome}}(T1)$

- **Commenti:** La parte destra nell'espressione T1 è completamente preservata dal $\bowtie_{\text{RIGHT}}$

Esercizio: Full outer-join bidirezionale

- Schema:

ARTISTA(AID, Nome)

ALBUM(AID, Titolo)

- **Traccia:** Produrre elenco combinato di Artista–Titolo, includendo artisti senza album e album senza artista registrato
- **Espressione algebrica:**

$T1 \leftarrow \text{ARTISTA} \bowtie_{\text{FULL ARTISTA.AID} = \text{ALBUM.AID}} \text{ALBUM}$

$\text{RISULTATO} \leftarrow \pi_{\text{Nome, Titolo}}(T1)$

- **Commenti:** $\bowtie_{\text{FULL}}$ unisce le due relazioni e conserva entrambe le parti orfane con colonne NULL

Esercizio: Differenza insiemistica per “NOT IN”

- Schema:

CLIENTE(CID, Nome)

FATTURA(CID, Totale)

- **Traccia:** Trovare i clienti che non hanno **alcuna** fattura registrata
- **Espressione algebrica:**

$$\pi_{CID, Nome} (CLIENTE) - \pi_{CID, Nome} (CLIENTE \bowtie FATTURA)$$

- **Commenti:** La differenza insiemistica restituisce esattamente i clienti orfani di fattura

Esercizio: Aggregazione con HAVING

- Schema:

VOTO(Matr, CodCorso, Punti)

- **Traccia:** Elencare CodCorso con media punti **superiore a 27**
- **Espressione algebrica:**

$\sigma_{\text{Media} > 27}(\gamma_{\text{CodCorso}; \text{AVG}(\text{Punti}) \rightarrow \text{Media}}(\text{VOTO}))$

- **Commenti:** σ sopra γ modella clausola HAVING; elimina corsi con media più bassa

Esercizio: Push-down su prodotto cartesiano

- Schema:

FORNITORE(FID, Paese)

COMPONENTE(CID, Peso)

- **Traccia:** Produrre tutte le coppie (Fornitore, Componente) in cui Paese = "DE" e Peso < 10; evitare il prodotto completo
- **Espressione algebrica:**

$$\sigma_{\text{Paese}='DE'}(\text{FORNITORE}) \times \sigma_{\text{Peso} < 10}(\text{COMPONENTE})$$

- **Commenti:** Filtri applicati prima del $\times$ riducono il numero di combinazioni

Esercizio: Vista virtuale espansa

- Schema:

IMP(Matricola, Nome, Laurea)

- Vista in SQL:

```
CREATE VIEW Laureati AS
```

```
SELECT Matricola, Nome FROM IMP WHERE Laurea = 'SI';
```

- **Traccia:** Tradurre in algebra relazionale la query `SELECT Nome FROM Laureati`
- **Espressione algebrica:**

$$\pi_{\text{Nome}} (\pi_{\text{Matricola}, \text{Nome}} (\sigma_{\text{Laurea}='SI'} (\text{IMP})))$$

- **Commenti:** La vista Laureati è sostituita con la propria definizione e semplificata con la π esterna

Esercizio: Outer-join e logica a tre valori (3VL)

- Schema:

UTENTE(UID, Nome)

LOG(LOGID, UID, Esito) – esito: 'OK' o NULL (mancato tentativo)

- **Traccia:** Mostrare tutti gli utenti e l'esito dell'ultimo log; se non esiste log, colonne a NULL
- **Espressione algebrica:**

$$T \leftarrow \gamma_{\text{UID}; \text{MAX}(\text{LOGID}) \rightarrow \text{Last}}(\text{LOG})$$
$$\text{ULAST} \leftarrow \text{UTENTE} \bowtie_{\text{LEFT UTENTE.UID} = \text{T.UID}} T$$
$$R \leftarrow \text{ULAST} \bowtie_{\text{LEFT ULAST.UID} = \text{LOG.UID} \wedge \text{ULAST.Last} = \text{LOG.LOGID}} \text{LOG}$$
$$\pi_{\text{UID, Nome, Esito}}(R)$$

- **Commenti:** Due $\bowtie_{\text{LEFT}}$ garantiscono che ogni utente appaia; il secondo join riporta l'esito o NULL se inesistente, rispettando la logica 3VL

Riepilogo

- 
- **Operatori avanzati introdotti:** Divisione ($\div$), aggregazione γ , outer-join ($\bowtie_{\text{LEFT}}$, $\bowtie_{\text{RIGHT}}$, $\bowtie_{\text{FULL}}$), logica a tre valori (NULL), bag-semantics essenziale
 - **Traduzione da SQL ad algebra relazionale:** SELECT / FROM / WHERE / GROUP BY / HAVING mappati su π , σ , $\bowtie$, γ ; sottoquery senza nomi speciali riscritte con join, proiezioni e differenze
 - **Viste:** Espressioni di algebra relazionale denominate; possono rimanere virtuali o essere espanse per l'esecuzione
 - **Ottimizzazione cost-based:**
 - Stime di cardinalità, scelta dell'ordine dei join (left-deep vs bushy)
 - σ e π spinti verso le foglie riducono i dati intermedi
 - **Esercizi pratici:** tutti gli operatori visti: universali, aggregazioni, outer-join, differenze, push-down
 - **Messaggio-chiave:** L'algebra relazionale estesa copre le esigenze reali di SQL e fornisce il linguaggio formale su cui il DBMS basa ottimizzazione e piani di esecuzione